DATA SCIENCE & DATA ANALYTICS USING PYTHON

TASK 1 — EXPANDED PROJECT GUIDE

Retail Sales Data Analysis and Interactive Dashboard

Detailed explanation of the concepts, implementation, formulas, code workflow, interpretation, validation, submission and interview preparation.

Dataset disclosure: The bundled dataset is synthetic practice data created to demonstrate cleaning and analysis. All metrics and rankings are examples, not real company results.

1. Understanding the internship task

The supplied internship brief defines Task 1 as a foundation-building task. It covers Python programming, mathematics, statistics, NumPy, Pandas, data collection, data cleaning, exploratory data analysis (EDA), visualization and a retail-sales mini project. Its expected deliverables include source code, notebooks, cleaned data, visualizations, an EDA report, GitHub repository, README, screenshots and documentation.

What the finished project should demonstrate

• Import a dataset and inspect its structure before analysis.

• Identify and document missing values, duplicates, invalid types and unusual records.

• Use Python, NumPy and Pandas to calculate summaries and derived metrics.

• Answer business questions with tables and charts.

• Explain what the findings mean, what assumptions were made and what limitations remain.

• Organize the work so another person can reproduce it.

2. Business problem and analytical questions

A retail business wants to understand sales performance. The project uses order date, customer, region, product, category, quantity, unit price, discount and payment method. It calculates gross sales and discounted revenue.

• Which product generates the highest total revenue?

• Which region contributes the most revenue?

• How does revenue change month by month?

• Which customers contribute the most revenue?

• Are there data-quality issues that could distort the results?

• How are quantity, price, discount, gross sales and revenue related?

A good analyst does not jump directly to charts. First confirm column definitions, units, date range, uniqueness of order identifiers, and whether each row represents an order, an order line, or a transaction. This sample treats each row as a sales line.

3. Project structure and files

```
retail_sales_data_analysis_task1/
■■■ datasets/
■   ■■■ raw/retail_sales_raw.csv
■   ■■■ cleaned/retail_sales_clean.csv
■■■ notebooks/retail_sales_analysis.ipynb
■■■ scripts/analyze_retail_sales.py
■■■ reports/
■   ■■■ eda_report.md
■   ■■■ summary_metrics.json
■   ■■■ figures/
■■■ documentation/TASK1_FULL_EXPLANATION.pdf
■■■ requirements.txt
■■■ .gitignore
■■■ README.md
```

Raw data is preserved separately from cleaned data so the original source remains auditable. The script holds repeatable analysis logic; the notebook provides an interactive explanation; reports store results; figures store exported charts.

4. Python programming fundamentals

Variables, types and collections

Variables bind names to objects. Typical types: int for whole numbers, float for decimal numbers, str for text and bool for true/false values. Lists are ordered and mutable; tuples are ordered and immutable; sets contain unique elements; dictionaries map keys to values.

```
order_id = "ORD-0001"
quantity = 2
unit_price = 1200.0
discount = 0.10
```

```
is_paid = True
products = ["Keyboard", "Monitor"]
order = {"quantity": quantity, "unit_price": unit_price}
```

Operators and control flow

Arithmetic operators calculate values; comparison operators return Boolean results; logical operators combine conditions. if/elif/else selects a path, and loops repeat a task. Use conditions to validate values before calculating.

```
gross_sales = quantity * unit_price
revenue = gross_sales * (1 - discount)
if quantity <= 0:
    print("Invalid quantity")
elif revenue >= 2000:
    print("Higher-value line")
else:
    print("Standard-value line")
```

Functions, modules and exceptions

A function is a named reusable block. Parameters make it flexible; a return value sends the result back. Modules group related functions. Exceptions should be handled narrowly and meaningfully instead of silently ignoring errors.

```
def calculate_revenue(quantity, unit_price, discount=0.0):
    if quantity <= 0:
        raise ValueError("Quantity must be positive")
    if unit_price < 0 or not 0 <= discount <= 1:
        raise ValueError("Invalid price or discount")
    return quantity * unit_price * (1 - discount)

try:
    print(calculate_revenue(2, 1200, 0.10))  # 2160.0
except ValueError as exc:
    print("Check input:", exc)
```

File handling and object-oriented programming

Use pathlib for portable paths and context managers for safe file handling. OOP defines classes that combine data and behavior. A Product class might hold a product name and price; for this analysis, straightforward functions and DataFrames are sufficient.

```
from pathlib import Path
Path("notes.txt").write_text("Sales analysis", encoding="utf-8")
print(Path("notes.txt").read_text(encoding="utf-8"))

class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
```

5. Mathematics for data science

Percentages and ratios

Percentage change = (new value – old value) / old value × 100. A ratio compares two quantities. Discount fraction 0.10 means 10%; multiplying gross sales by (1 – discount) gives net revenue before any other adjustments.

```
old, new = 1000, 1250
change_pct = (new - old) / old * 100
print(change_pct)  # 25.0
```

Linear equations, vectors and matrices

A linear equation expresses a relationship between variables. In the simplest sales calculation, gross sales = quantity × unit price. A vector is a one-dimensional collection; a matrix is a two-dimensional arrangement. NumPy performs element-wise vector operations efficiently.

```
import numpy as np
quantity = np.array([2, 3, 1])
price = np.array([1200, 650, 12500])
print(quantity * price)  # [2400 1950 12500]
matrix = np.array([[1, 2], [3, 4]])
vector = np.array([10, 20])
print(matrix @ vector)
```

Logarithms and introductory calculus

Logarithms can compress values spanning large ranges and appear in growth, probability and machine-learning methods. Calculus studies change and accumulation. A derivative describes the local rate of change; optimization methods use derivatives to adjust model parameters. These ideas are foundational rather than directly required for the current sales

summaries.

6. Statistics

Mean, median and mode

Mean is the sum divided by count. Median is the middle observation after sorting and is often less affected by extreme values. Mode is the most frequent value. Comparing mean and median can reveal skew.

Variance and standard deviation

Variance summarizes squared deviations from the mean. Standard deviation is its square root and is in the same units as the data. NumPy's var() and std() use population calculations by default (ddof=0); sample estimates commonly use ddof=1.

```
values = np.array([100, 120, 120, 140, 220])
print("Mean:", values.mean())
print("Median:", np.median(values))
print("Population std:", values.std())
print("Sample std:", values.std(ddof=1))
```

Probability, distributions, covariance and correlation

Probability quantifies uncertainty from 0 to 1. A distribution describes possible values and their frequency or probability. Covariance shows whether two variables tend to move together but depends on their units. Correlation standardizes linear association to the range –1 to +1. Correlation does not establish causation.

7. NumPy essentials

NumPy arrays store homogeneous data efficiently and support vectorization, broadcasting, indexing, slicing and mathematical functions. Broadcasting allows compatible shapes to operate without writing explicit Python loops.

```
arr = np.array([[10, 20, 30], [40, 50, 60]])
print(arr.shape)     # (2, 3)
print(arr[0, 1])     # 20
print(arr[:, 1:])    # columns 2 and 3 for all rows
print(arr * 0.9)     # apply a 10% reduction
rng = np.random.default_rng(42)
print(rng.integers(1, 6, size=5))
```

Indexing selects positions; slicing selects ranges; random generators create reproducible sample values when a seed is supplied. Vectorized operations usually produce shorter and faster numerical code.

8. Pandas essentials

Series and DataFrames

A Series is a one-dimensional labeled structure; a DataFrame is a table with rows and columns. read_csv() and read_excel() import files. head(), info(), describe(), shape and dtypes help inspect the data.

```
import pandas as pd
df = pd.read_csv("datasets/raw/retail_sales_raw.csv")
print(df.shape)
print(df.head())
df.info()
print(df.describe(include="all").T)
```

Filtering, sorting, groupby, merge, join and pivot

```
east_sales = df[df["Region"] == "East"]
expensive = df.sort_values("Unit_Price", ascending=False).head(10)
by_product = df.groupby("Product", dropna=False)["Quantity"].sum()
# Example when a second lookup table exists:
# merged = orders.merge(product_lookup, on="Product_ID", how="left")
# A pivot summarizes a measure across categories:
pivot = pd.pivot_table(df, values="Quantity", index="Region",
                       columns="Category", aggfunc="sum", fill_value=0)
```

groupby divides records into groups and applies an aggregation. merge combines tables using columns or keys. join commonly aligns by index, though it can also use a key. pivot_table reshapes a table into a summarized matrix.

9. Data collection and source quality

The supplied brief lists CSV, Excel, JSON, public datasets, Kaggle and government open data. Before using external data, check the license, provenance, privacy restrictions, date coverage, definitions and whether the data represents the question.

• Kaggle datasets: https://www.kaggle.com/datasets

• UCI Machine Learning Repository: https://archive.ics.uci.edu/

• Data.gov: https://data.gov/

• Google Dataset Search: https://datasetsearch.research.google.com/

• World Bank Open Data: https://data.worldbank.org/

• Our World in Data: https://ourworldindata.org/

10. Data cleaning: exact decisions in this project

The script strips column-name whitespace, parses Order_Date as a date, converts quantity/price/discount to numeric, removes exact duplicate rows, excludes rows with missing or non-positive quantity, labels missing categories as Unknown, fills missing discount with zero, clips discount to the 0–1 range, and excludes missing or negative unit prices.

```
df.isna().sum()                 # missing values by column
df.duplicated().sum()           # exact duplicates
df = df.drop_duplicates()
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
```

Why document these rules? Missing quantity prevents reliable revenue calculation. Missing region is labelled Unknown rather than guessed. Filling a missing discount with zero is an assumption used for demonstration and should be confirmed with the business owner in real work. Outliers should be investigated, not automatically deleted: they may be data errors or legitimate high-value transactions.

11. Feature engineering and revenue formula

```
Gross_Sales = Quantity * Unit_Price
Revenue = Gross_Sales * (1 - Discount)
Month = Order_Date converted to calendar year-month
```

Example: quantity 2 × unit price ■1,200 = ■2,400 gross sales. With a 10% discount, revenue = ■2,400 × (1 – 0.10) = ■2,160. Revenue here means discounted line sales; it is not profit because cost of goods, shipping, tax, returns and operating expenses are not included.

12. Exploratory data analysis (EDA)

EDA is an iterative process for understanding quality, distribution, relationships and patterns before making decisions. Start with shape and types, then missingness and duplicates, descriptive statistics, group comparisons, monthly trends and correlation. Write down observations and their limitations.

```
print(clean.describe(numeric_only=True).T)
print(clean.groupby("Product")["Revenue"].agg(["sum", "mean", "count"]))
print(clean.groupby("Region")["Revenue"].sum().sort_values(ascending=False))
print(clean.groupby("Month")["Revenue"].sum())
```

13. Visualizations and when to use them

• Bar chart: compare revenue across products or regions; sort bars to improve readability.

• Line chart: display monthly revenue over time; ensure dates are sorted.

• Histogram: understand the distribution and skew of order-line revenue.

• Scatter plot: explore relationships between two numerical variables.

• Box plot: compare spread and flag possible outliers for investigation.

• Heatmap: show correlation values or a two-way summary table.

• Pie chart: use sparingly for a small number of parts of a whole; bars are easier for close comparisons.

Every chart should have a question, informative title, labeled axes, units and readable ticks. Visual patterns can suggest hypotheses but do not by themselves prove causes.

14. Project implementation and execution

Install dependencies

```
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Run the analysis

```
python scripts/analyze_retail_sales.py
```

The script reads datasets/raw/retail_sales_raw.csv, writes the cleaned file under datasets/cleaned, saves four figures in reports/figures, writes summary_metrics.json, and creates eda_report.md. The notebook is provided for cell-by-cell exploration. Run from the project root so relative paths resolve correctly.

15. Result interpretation from the bundled practice dataset

The generated analysis reported 352 raw rows, 345 cleaned rows, 2 exact duplicate rows removed, total calculated revenue of ■2,299,716.62, average retained row revenue of ■6,665.85, highest-revenue product Monitor and highest-revenue region East. The data spans 2025-01-01 through 2025-12-31.

Interpretation caution: The dataset was synthetically generated and deliberately includes data-quality issues. The rankings and amounts only validate the program's workflow; they must not be presented as real business findings. Also, average row revenue is not necessarily average order value if one order can contain multiple lines.

16. Business insights: how to reason responsibly

• Product ranking: inspect revenue, unit volume, discount and profit margin together before making inventory recommendations.

• Regional ranking: compare revenue with order count, customer count and average basket to separate volume from order size.

• Monthly trend: one year can suggest a pattern but is insufficient to establish long-term seasonality.

• Customer ranking: check repeat purchases and concentration risk; protect customer privacy.

• Underperforming category: compare against targets, margins, stock-outs and promotional exposure rather than revenue alone.

17. Quality checks and common errors

• KeyError for a column: print df.columns and check spelling/capitalization; normalize column names if needed.

• All arrays must be of the same length: make sure each dictionary column has the same number of values before constructing a DataFrame.

• Dates remain text: use pd.to_datetime(..., errors='coerce') and inspect rows that become NaT.

• Unexpected missing values after conversion: inspect the source strings and conversion failures before deciding how to handle them.

• Empty chart or wrong totals: verify filters, data types, missing-value handling and aggregation level.

• Path not found: run the command from the project root or use pathlib paths based on __file__.

• Chart package issues: use the installed dependencies from requirements.txt and check that the active Python environment matches the one where packages were installed.

18. GitHub publication

```
git init
git add .
git commit -m "Complete Task 1 retail sales analysis project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/retail-sales-data-analysis-task1.git
git push -u origin main
```

Create an empty repository on GitHub, replace YOUR-USERNAME with your username, and use the repository's actual URL. Do not commit passwords, access tokens, private datasets or personal customer information. A useful README includes overview, setup, data dictionary, cleaning decisions, chart screenshots, findings, limitations and future work.

19. Sample viva and interview answers

Q1. Why did you use Pandas?

Pandas provides labeled tables, file import, missing-value handling, filtering, grouping, joins and convenient summaries, which are well suited to tabular sales data.

Q2. How did you calculate revenue?

Gross sales are quantity multiplied by unit price. Revenue is gross sales multiplied by one minus the discount fraction. A 0.10 discount means 10%.

Q3. How did you handle missing values?

I inspected missing counts first. Rows with missing or invalid quantity were excluded because revenue could not be calculated reliably. Missing categorical fields were labeled Unknown, and missing discount was set to zero as a documented sample-data assumption.

Q4. What is the difference between correlation and causation?

Correlation measures association between variables. It does not establish that one variable caused another; confounders, selection effects and other explanations may exist.

Q5. What are the limitations?

The data is synthetic, covers only one year, and does not include product cost, returns, tax, shipping or profit. Its results demonstrate code behavior rather than business reality.